

MSNoise

Note:

The conda environment 'msnoise' was based on python 3.7. MSNoise Admin, the web interface component of MSNoise expects higher version of python. Also, the version of MSNoise installed relies on older versions of the Flask package which earlier had aliases to markupsafe which is now a separate package. So, I created a new environment with python 3.9 and downgraded Flask.

```
$ conda create -n msnoise_new python=3.9
$ conda activate msnoise_new
$ conda install -c conda-forge obspy
$ pip install msnoise
$ pip install "Flask<3.0" "Werkzeug<3.0" "markupsafe<2.1"
```

I have also added an alias to msnoise in **.bashrc** as every operation in MSNoise follows \$ msnoise ...

```
$ nano ~/.bash rc
alias ms=msnoise
^O ^X
```

Thus, before starting to work, run:

```
$ conda activate msnoise_new
```

Initializing Project

\$ ms db init [To be run in the project folder]

- This command creates a database for MSNoise to work on.

- Database can be generated with two different backends:

- 1) SQLite: A local database in the project folder.
- 2) MySQL: Connect to a local or remote mysql server, additional information is then required:
 - hostname: of the mysql server, defaults to 127.0.0.1
 - database: must already exist on hostname
 - username: as registered in the privileged users of the mysql server
 - password: his password

- Table Prefix: During initialization, MSNoise allows us to define a Table Prefix (e.g., proj_1). This prefix is prepended to all database tables. It allows us to run multiple independent MSNoise projects on a single SQL database instance without naming conflicts. For general use, enter blank.

- Output Files:

- 1) db.ini: Stores the information of connecting to the database.
- 2) msnoise.sqlite: The physical database file created only if SQLite is chosen as the backend. It stores all metadata tables including stations, scanned data, jobs, filters, and config sets.

- Both these files must remain in the project folder where we will run all CLI commands.

Web Interface

\$ ms admin

- Launches a web-based administration interface
- By default, it starts the web server on port 5000 and can be accessed on <http://localhost:5000/>
- The default port can be overridden by passing parameters to the command, e.g. for port 5099:

\$ ms admin -p 5099

- The Home Page gives an overview of the data available, jobs in database and the progress.

The screenshot shows the MSNoise Dashboard web interface in a browser. The address bar shows <http://localhost:5000/admin/>. The navigation bar includes links for MSNoise, Home, Configuration, Database, Results, and Help. The main content area is titled "MSNoise Dashboard" and displays the following information:

- Project Folder: /home/arjun/Anubhab Sen/Data/Control
- Project Database: SQLite: msnoise.sqlite
- Status**
- Data Availability Scan**: 79207 files identified in the archive: 0 new, 0 modified and 79207 archived. Progress bar at 100%.
- Cross-Correlation Jobs**: Buttons for "Reset CC jobs 'in progress'", "Reset all CC jobs", and "Delete all CC jobs". 489 CC jobs in the database: 9 todo, 1 in progress and 479 done. Progress bar at 97%.
- Stack Jobs**: Buttons for "Reset STACK jobs 'in progress'", "Reset all STACK jobs", and "Delete all STACK jobs". 479 STACK jobs in the database: 479 todo, 0 in progress and 0 done.
- MWCS Jobs**: Buttons for "Reset MWCS jobs 'in progress'", "Reset all MWCS jobs", and "Delete all MWCS jobs". 0 MWCS jobs in the database: 0 todo, 0 in progress and 0 done.
- Relative Travel Time Jobs**: Buttons for "Reset DTT jobs 'in progress'", "Reset all DTT jobs", and "Delete all DTT jobs". 0 DTT jobs in the database: 0 todo, 0 in progress and 0 done.
- Actions ?**: Buttons for "Check for New Jobs" and "Shutdown Admin Web Interface".

- Database Tables
 - Stations: stores station codes, networks, coordinates (latitude, longitude, elevation), and status.
 - Filters: Defines the frequency bands used for processing along with corresponding window lengths for cross-correlations and MWCS.
 - Data Availability: Dynamically updated by archive scans; tracks which days have data for each station.
 - Jobs: The state-aware scheduler queue mapping every day, station pair, and workflow step to progress flags.
 - Config: Holds global key-value configuration parameters.
- Core Configuration Fields
 - Data Folder: The root directory containing your raw seismic waveform archives.
 - Data Structure: Defines how your archive folders are nested.
 - Network & Channel: Regular expressions or filters to selectively scan certain components or networks (e.g., target only HHZ channels to avoid transverse component processing).
 - Start Date & End Date: The definitive operational window for your project. No files or jobs outside this range will be processed.
- All the parameters for computations can also be set in the **Config** menu in the **Configuration** tab.
- Upon hosting MSNoise Admin, the terminal remains busy until the server is closed using Keyboard Interrupt (Ctrl+C). In the meantime, another terminal window should be opened in the project folder for further commands.

Populating Station Table

- MSNoise needs to have the data of stations for cross correlating their data.
- The MSEED files must be stored in a directory in a specific format to be called the Data Structure.
- MSNoise has few pre-defined data structures:

```
data_structure['SDS'] = "YEAR/NET/STA/CHAN.TYPE/NET.STA.LOC.CHAN.TYPE.YEAR.DAY"
data_structure['BUD'] = "NET/STA/STA.NET.LOC.CHAN.YEAR.DAY"
data_structure['IDDS'] = "YEAR/NET/STA/CHAN.TYPE/DAY/NET.STA.LOC.CHAN.TYPE.YEAR.DAY.HOUR"
data_structure['PDF'] = "YEAR/STA/CHAN.TYPE/NET.STA.LOC.CHAN.TYPE.YEAR.DAY"
```

- However, we can define any custom data structure for our project.
- For example, the data structure of WESTERN TIBET files is: **STA/NET.STA.YEAR.DAY.HOUR.CHAN**
- To specify the Data Structure using MSNoise Admin, open the **Config** menu from the **Configuration** tab. Find the parameter **data_structure** and input the desired data structure.
- If the data structure is any of the pre-defined data structures, the acronym (SDS, BUD, IDDS or PDF) can be used. In case of custom data structure, the entire path (STA/NET.STA.YEAR.DAY.HOUR.CHAN) has to be written.
- In case of a custom data structure, MSNoise expects to find a file named **custom.py** in the current folder. This python file should contain a function called **populate** which would accept one argument (path to root directory) and return a station dictionary with keys of the format **NET_STA**, and fields for the stations table in the database: Network, Station, Longitude, Latitude, Altitude, Coordinates(UTM/DEG), Instrument.

Note: In our data from WESTERN TIBET, the MSEED files do not encode the station details such as Longitude, Latitude or Altitude. Therefore, we include these data while populating station table, we need to have a file containing these information which can be read by custom.py.

- For example, writing **custom.py** for WESTERN TIBET: [custom.py](#)
- Station coordinates are contained in **station_info.txt**: [station_info.txt](#)
- After having these files in the project folder, following command can be run:

\$ ms populate

- It scans the root directory and identifies the networks and stations using the given data structure.
- If a custom data structure is used, custom.py runs.

MSNoise Home Configuration Database Results Help									
List (4)	Create	Add Filter	20 items	With selected					
☐		Net	Sta	X	Y	Altitude	Coordinates	Instrument	Used
☐	 	Y2	WT01	81.925598	32.042702	4600.0	UTM	N/A	
☐	 	Y2	WT02	81.364998	30.759701	4693.0	UTM	N/A	
☐	 	Y2	WT08	80.544502	31.356199	4807.0	UTM	N/A	
☐	 	Y2	WT11	81.356697	32.224201	4848.0	UTM	N/A	

- The above steps of creating custom data structure can be skipped using the following command:

```
$ ms scan_archive --path path/to/root/directory --init --recursively
$ ms populate --fromDA
```

- However, this can work only if MSEED files encode the coordinates of the stations.

Scanning Archive

```
$ ms scan_archive --path path/to/root/directory --init --recursively
```

- Scans through the root directory, passed using **--path** parameter and inserts records into the data_availability table.

- The **--init** parameter performs a fresh complete index scan. Without passing the parameter, only the files which were added to the directory in last 1 day would be scanned. This duration can be modified by changing the value of the parameter **crondays** in the **Config** menu in **Configuration** tab of **MSNoise** Admin. Thus, for re-scanning a directory after addition of new MSEED files, the following command can be used to save time:

```
$ ms scan_archive --path path/to/root/directory --recursively
```

- The --recursively parameter visits all subfolders nested inside the targetted directory in the search of MSEED files. Thus, if a directory with no sub folders has to be scanned:

\$ ms scan_archive --path path/to/directory

- After scanning archive, station tables can be populated directly using:

\$ ms populate --fromDA

-However, this would only work if the MSEED files encode the coordinates of the stations.

MSNoise Home Configuration Database Results Help												
List (79207)		Add Filter		100 items		With selected						
		Net	Sta	Comp	Path	File	Starttime	Endtime	Data Duration	Gaps Duration	Samplerate	Flag
☐		Y2	WT01	HHE	/home/arjun/ Anubhab Sen/ Data/WT01	Y2.WT01.2011.058.04.HHE.MSEED	2011-02-27 04:00:00	2011-02-27 04:59:59	3600.0	0.0	100.0	A
☐		Y2	WT01	HHZ	/home/arjun/ Anubhab Sen/ Data/WT01	Y2.WT01.2011.058.04.HHZ.MSEED	2011-02-27 04:00:00	2011-02-27 04:59:59	3600.0	0.0	100.0	A
☐		Y2	WT01	HHN	/home/arjun/ Anubhab Sen/ Data/WT01	Y2.WT01.2011.058.04.HHN.MSEED	2011-02-27 04:00:00	2011-02-27 04:59:59	3600.0	0.0	100.0	A
☐		Y2	WT02	HHE	/home/arjun/ Anubhab Sen/ Data/WT02	Y2.WT02.2011.030.07.HHE.MSEED	2011-01-30 07:00:00	2011-01-30 07:59:59	3600.0	0.0	100.0	A
☐		Y2	WT02	HHE	/home/arjun/ Anubhab Sen/ Data/WT02	Y2.WT02.2010.281.00.HHE.MSEED	2010-10-08 00:00:00	2010-10-08 01:00:00	3600.01	0.0	100.0	A
☐		Y2	WT02	HHN	/home/arjun/ Anubhab Sen/ Data/WT02	Y2.WT02.2011.016.21.HHN.MSEED	2011-01-16 21:00:00	2011-01-16 22:00:00	3600.01	0.0	100.0	A
☐		Y2	WT02	HHE	/home/arjun/ Anubhab Sen/ Data/WT02	Y2.WT02.2010.347.20.HHE.MSEED	2010-12-13 20:00:00	2010-12-13 20:59:59	3600.0	0.0	100.0	A
☐		Y2	WT02	HHZ	/home/arjun/ Anubhab Sen/ Data/WT02	Y2.WT02.2011.030.09.HHZ.MSEED	2011-01-30 09:00:00	2011-01-30 09:59:59	3600.0	0.0	100.0	A
☐		Y2	WT02	HHZ	/home/arjun/ Anubhab Sen/ Data/WT02	Y2.WT02.2010.276.05.HHZ.MSEED	2010-10-03 05:00:00	2010-10-03 05:59:59	3600.0	0.0	100.0	A
☐		Y2	WT02	HHE	/home/arjun/ Anubhab Sen/ Data/WT02	Y2.WT02.2011.080.15.HHE.MSEED	2011-03-21 15:00:00	2011-03-21 15:59:59	3600.0	0.0	100.0	A
☐		Y2	WT02	HHE	/home/arjun/ Anubhab Sen/ Data/WT02	Y2.WT02.2010.273.17.HHE.MSEED	2010-09-30 17:00:00	2010-09-30 17:59:59	3600.0	0.0	100.0	A
☐		Y2	WT02	HHN	/home/arjun/ Anubhab Sen/ Data/WT02	Y2.WT02.2011.058.03.HHN.MSEED	2011-02-27 03:00:00	2011-02-27 03:59:59	3599.9900000000002	0.0	100.0	A
☐		Y2	WT02	HHN	/home/arjun/ Anubhab Sen/ Data/WT02	Y2.WT02.2010.260.18.HHN.MSEED	2010-09-17 18:00:00	2010-09-17 19:00:00	3600.01	0.0	100.0	A
☐		Y2	WT02	HHZ	/home/arjun/ Anubhab Sen/ Data/WT02	Y2.WT02.2010.292.15.HHZ.MSEED	2010-10-19 15:00:00	2010-10-19 15:59:59	3600.01	0.0	100.0	A

Initiatializing Jobs

- Before initiating jobs, a filter must be defined, as it determines the frequency bands in which the cross-correlations and velocity changes dv/v will be calculated.
- A filter can be defined in the **Filter** menu in the **Configuration** tab in MSNoise Admin. On clicking **Create**, the following parameters must be defined:
 - Ref (ID): Usually an integer index (e.g., 1, 2, 3). This is used to identify the filter in the database.
 - Low : The lower corner frequency of the bandpass filter.
 - High : The upper corner frequency of the bandpass filter.
 - MWCS Low : The lower frequency for the Moving Window Cross-Spectral analysis. Often identical to the filter's "Low" value.
 - MWCS High : The upper frequency for the MWCS analysis.
 - RMS Threshold: Defines the limit for the signal-to-noise ratio.
 - MWCS Window Length: The length of the sliding window (in seconds) used while MWCS analysis.
 - MWCS Step: The overlap/step between windows.
 - The Used parameter must be checked in order to apply the filter. It can be toggled off to ignore the filter in subsequent jobs.

- After defining a filter, the following command should be run:

\$ ms new_jobs --init

- Populates the `jobs` table by cross-referencing the `data_availability` table. It creates a task list for cross-correlation (CC), stacking (STACK), and delay-time calculations (MWCS).
- Each job in the queue has a status code tracked in the database:
 - T (Todo): Marked for processing. Eligible to be picked up by the next available worker thread.
 - I (In progress): A worker is currently processing this job. This prevents multi-threaded workers from computing duplicate files.
 - D (Done): Successfully completed.
 - E (Error): Process encountered an exception (e.g., corrupted MSEED file).

- Upon first run, if you expect the number of jobs to be large (many days, many stations), pass the `--init` parameter to optimize the insert. Only use this flag once, otherwise problems will arise from duplicate entries in the jobs table. From subsequent runs:

\$ ms new_jobs

Automating the Pre-processing of Data

- I have created a Python script that would input the path of the root directory, create a project folder, initiate database, write `custom.py`, polulate stations and `data_availability` tables, defines a default filter, initiates jobs and finally launches MSNoise Admin and a new terminal in project folder to work on.

/home/arjun/Anubhab Sen/PyCharm/Noise/Process.py: [Process.py](#)

- I have also added a script in bash that would run this script with a single command **\$ process:** [process script](#)

- For the ease of defining Data Structures, a file **Structures** has been created in **/home/arjun/Anubhab Sen/PyCharm/Noise** which stores all the predefined data structures and the one used in **WESTERN-TIBET** directory.

/home/arjun/Anubhab Sen/PyCharm/Noise/Structures: [Structures](#)

- The **custom.py** to be written by **process.py** would rely on **gmap-stations.txt** which it expects to find in the root directory.

~/WESTERN-TIBET/gmap-stations.txt : [gmap-stations.txt](#)

- Thus, before running the command, station details must be downloaded from <https://ds.iris.edu/gmap/> and saved as **gmap-stations.txt** in the root directory, without any modification of its contents.

- Thereafter, the command can be run:

\$ conda activate msnoise_new

\$ process --path path/to/root/directory/

-The command can also be run without the parameter **--path**. In this case, the terminal would ask for the root directory where its absolute path has to be entered.

\$ process

Enter the path of root directory: **/home/arjun/absolute/path/to/root/directory**

- Upon first run on a directory, a **Control** folder is created in the root directory and a new **db.ini** file would be initiated and terminal would ask for the data technology and table prefix, similar to running **\$ ms db init**

- Thereby, the program will ask for data structure as follows:-

Select Data Structure:

```
[1] SDS      : "YEAR/NET/STA/CHAN.TYPE/NET.STA.LOC.CHAN.TYPE.YEAR.DAY"
[2] BUD      : "NET/STA/STA.NET.LOC.CHAN.YEAR.DAY"
[3] IDDS     : "YEAR/NET/STA/CHAN.TYPE/DAY/NET.STA.LOC.CHAN.TYPE.YEAR.DAY.HOUR"
[4] PDF      : "YEAR/STA/CHAN.TYPE/NET.STA.LOC.CHAN.TYPE.YEAR.DAY"
[5] WESTERN_TIBET : "STA/NET.STA.YEAR.DAY.HOUR.CHAN" [Network: "Y2"]
[6] Create New Data Structure
```

Your Choice: **5**

- If a new data structure is created, it will be added to **Structures** and will be displayed in the list in subsequent runs.

- **custom.py** detects the network of stations dynamically if the data structure contains 'NET/'. However, if the network for all the stations are same, there might be no subfolder of class NET. In such case, the program would ask to enter the network manually, which would also be saved alongside the new data structure.

- The further steps are automated:

- 1) Saving the first run details in a file **info**
- 2) Creates a wide filter to take the entire data into account.
- 3) Writing **custom.py** for the given data structure.
- 4) Populating Stations table
- 5) Scans the root directory except the **Control** directory and the **All** directory, if present.
- 6) Initializes new jobs
- 7) Launches MSNoise Admin in a free localhost port.
- 8) Opens MSNoise Admin in Firefox and a terminal window in the **Control** directory.

- Upon subsequent runs,
 - 1) the program retrieves all the necessary information from the **info** file
 - 2) populates stations table
 - 3) scans for data which had been added after the last run
 - 4) inserts new jobs
 - 5) Launches MSNoise Admin in a free localhost port.
 - 6) Opens MSNoise Admin in Firefox and a terminal window in the **Control** directory.
- The program also tries to repair the files in **Control** folder in case they get corrupted or deleted.
- However, the program would terminate if any unavoidable Exception arises.
- The further commands for computing and plotting must be executed in the **Control** directory.

Computing Cross-Correlations

\$ ms compute_cc

- Computes cross-correlation functions (CCFs) for each station pair and outputs daily HDF5 files.

Stacking

- The cross-correlations can be stacked in three ways:
 - 1) -r computes the long-term Reference Stack representing the baseline, "undisturbed" medium state.
 - 2) -m computes the Moving Stack representing current states.
 - 3) -s is used in sub-daily stacks to evaluate variations on a smaller temporal resolution.

- Thus, to compute REF stack:

\$ ms stack -r

- Stacks are cumulative. If you update your station coordinates, add new filters, modify preprocessing configurations, or fix backfilled historical data, your previously calculated references are invalid. Running msnoise reset STACK flags all stacking jobs back to T (Todo), forcing the pipeline to recalculate stacks from raw CCFs.

- After computing REF stack, the stacking job must be reset back to Todo list before computing MOV stack:

\$ ms reset STACK

\$ ms stack -m

Moving Window Cross-Spectral Analysis

\$ ms compute_mcws

- Calculates phase delays (dt) between daily Moving Stacks and the long-term Reference Stack in the frequency domain. It windows the CCF coda into small, overlapping segments, measuring the phase shift of the cross-spectrum for each window to resolve travel-time delay.

Computing dt/t

\$ ms compute_dtt

- Performs a weighted linear regression (WLS) of the delays (dt) against the time-lags (t) of the coda windows:
 $dt/t = - dv/v$
- The slope of this line yields the relative velocity variation ($\dot{v}$), outputting the final time-series.

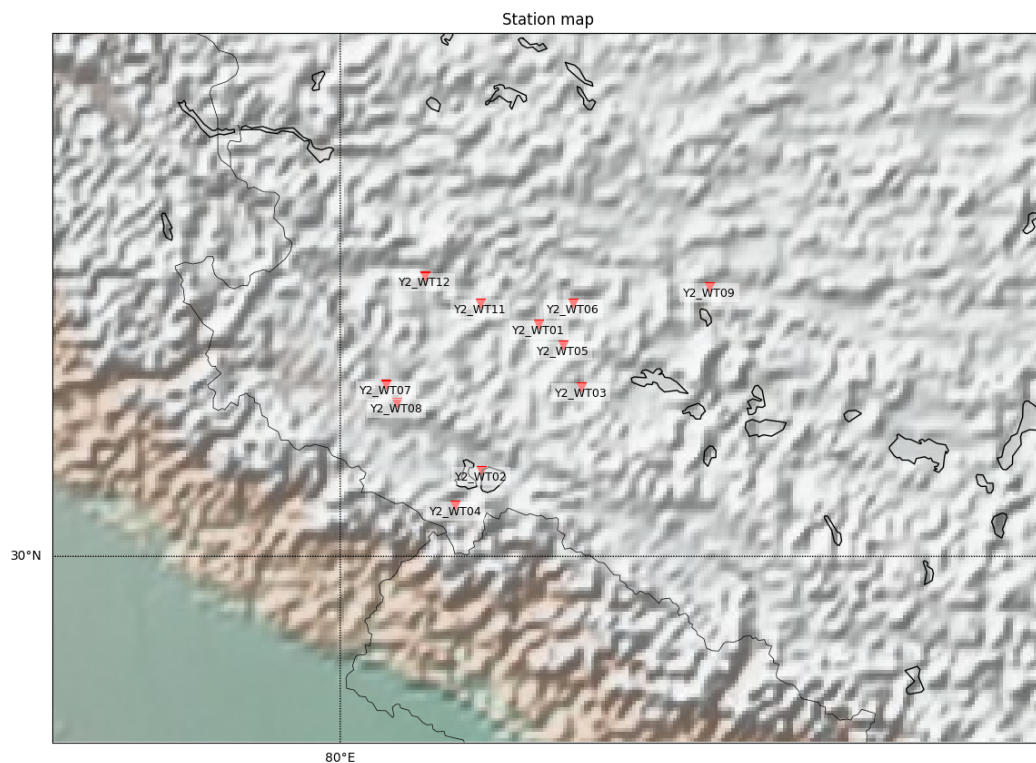
Visualizing Data

- The following commands can be used to plot graphs of the computed data:

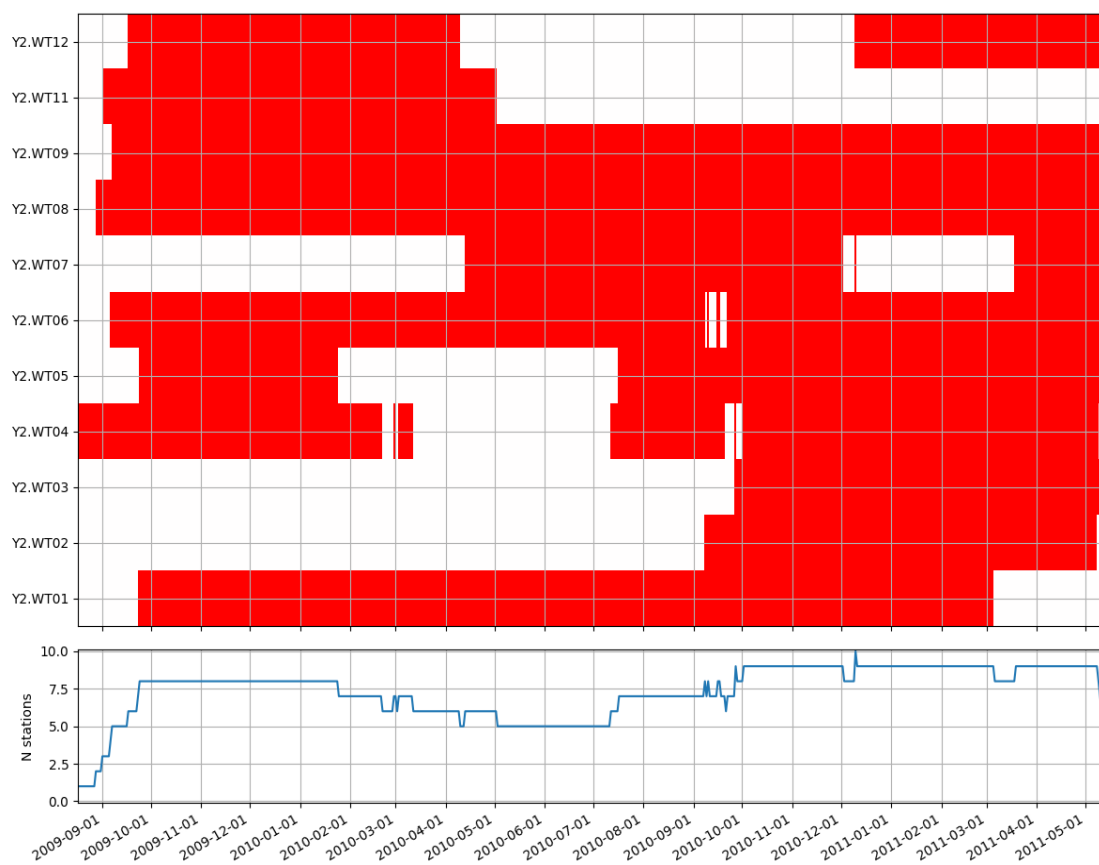
\$ ms plot --help

```
Commands:
ccftime      Plots the ccf vs time between sta1 and sta2
data_availability Plots the Data Availability vs time
distance     Plots the REFs of all pairs vs distance
dtt          Plots a graph of dt against t
dvv          Plots the dv/v (parses the dt/t results)
interferogram Plots the interferogram between sta1 and sta2...
mwcs         Plots the mwcs results between sta1 and sta2 (parses...
spectime     Plots the ccf's spectrum vs time between sta1 and sta2
station_map  Plots the station map (very very basic)
timing        Plots the timing (parses the dt/t results)
```

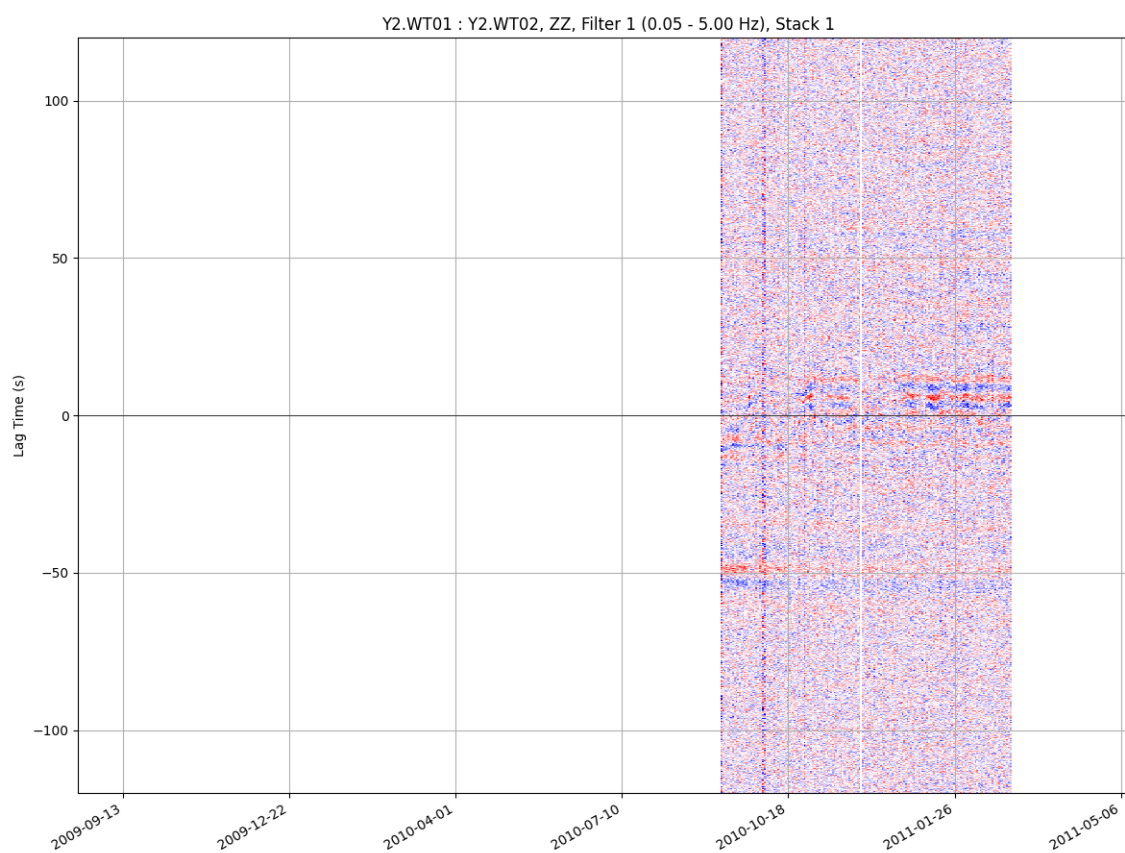
\$ ms plot station_map



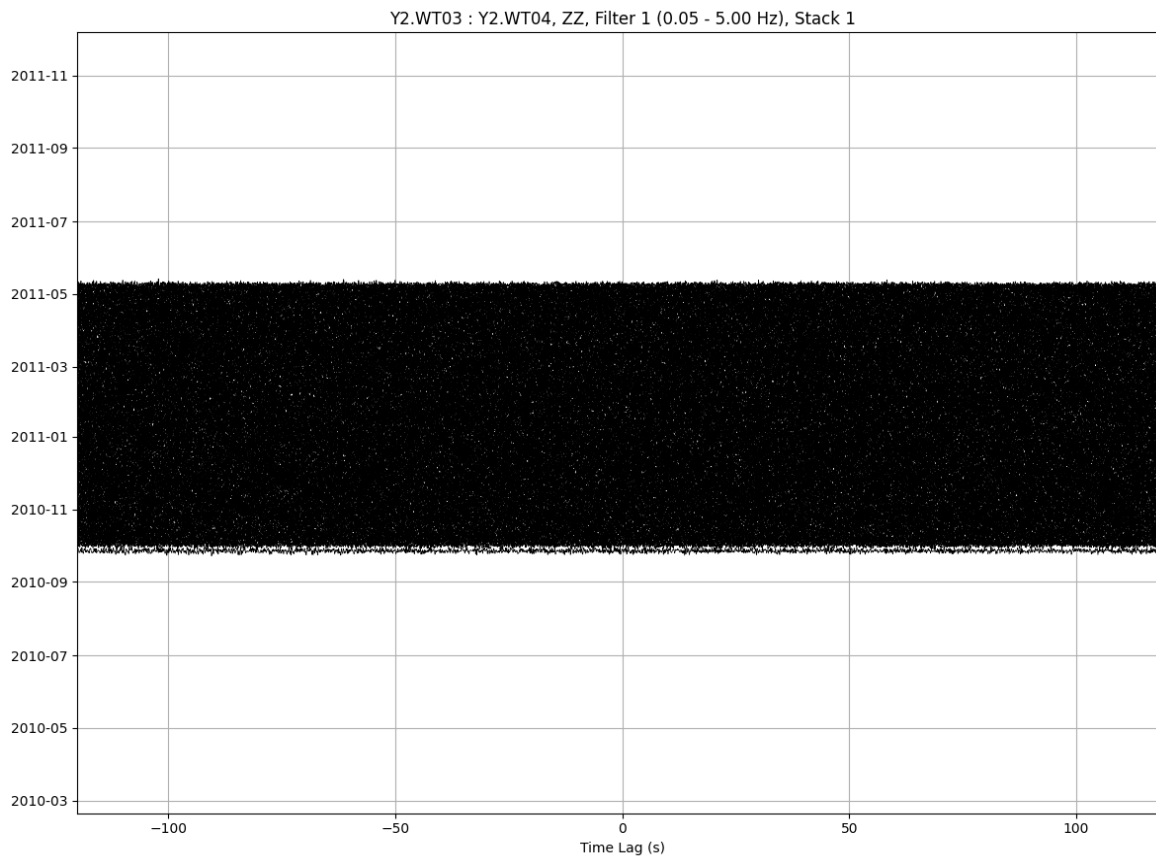
\$ ms plot data_availability



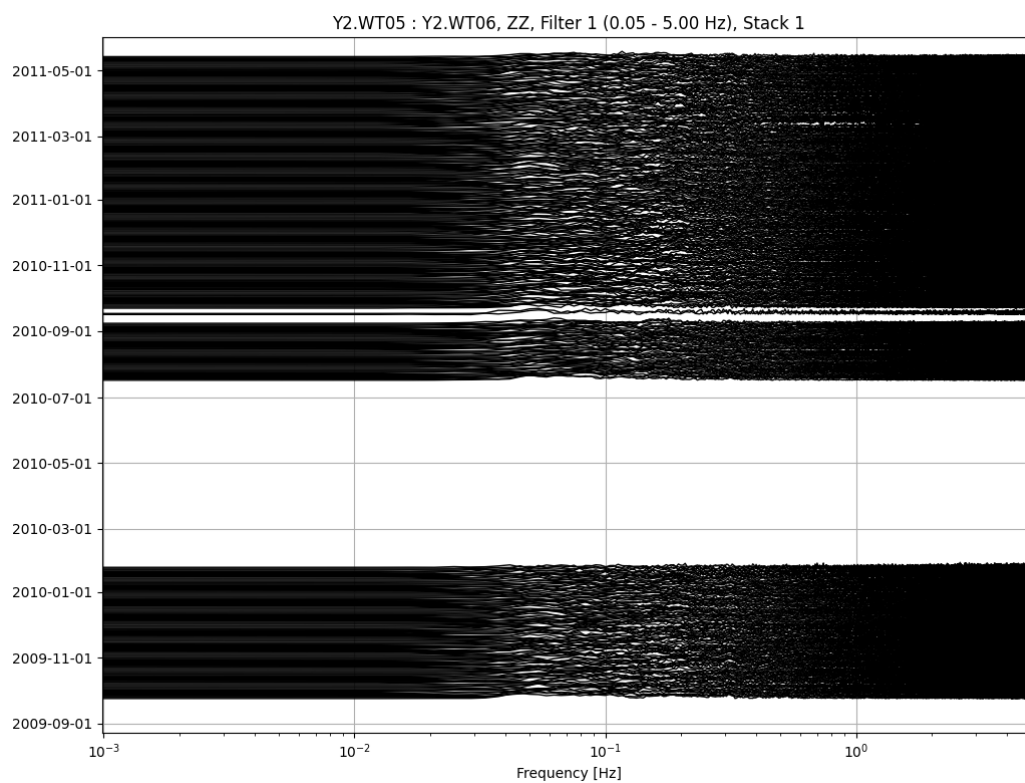
\$ ms plot interferogram Y2.WT01 Y2.WT02



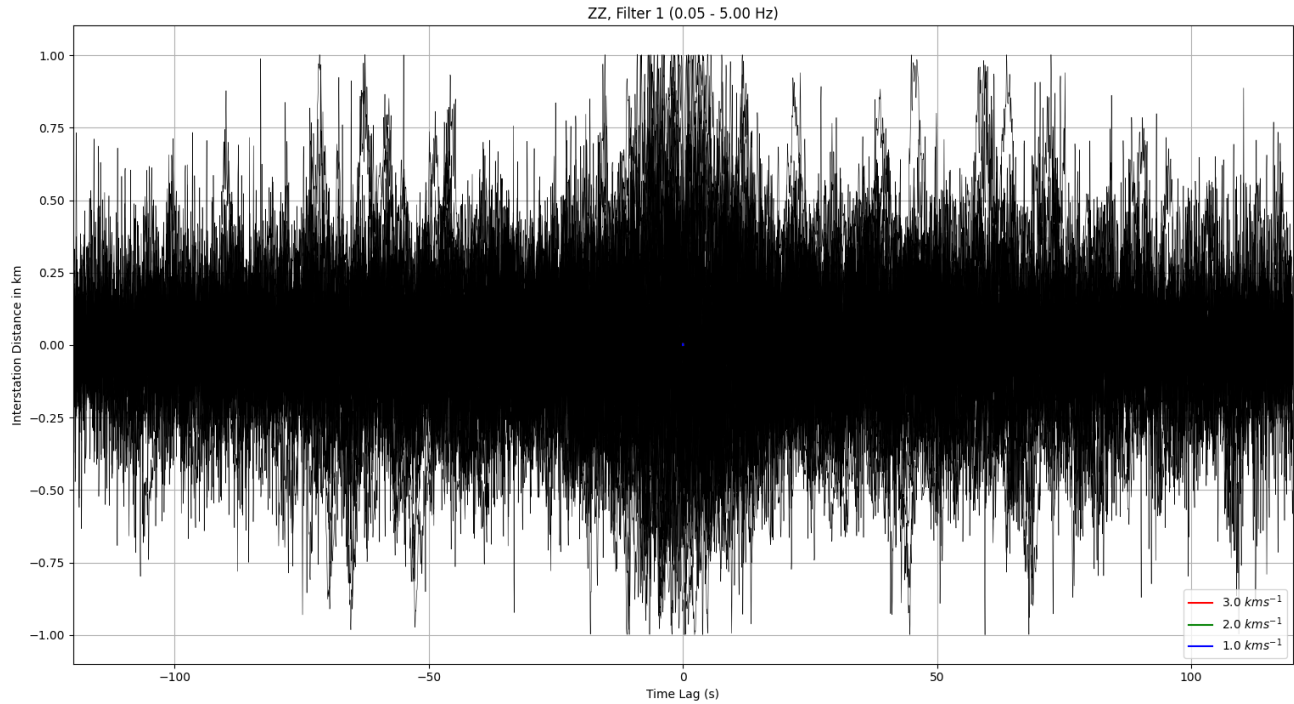
\$ ms plot ccftime Y2.WT03 Y2.WT04



\$ ms plot specttime Y2.WT05 Y2.WT06

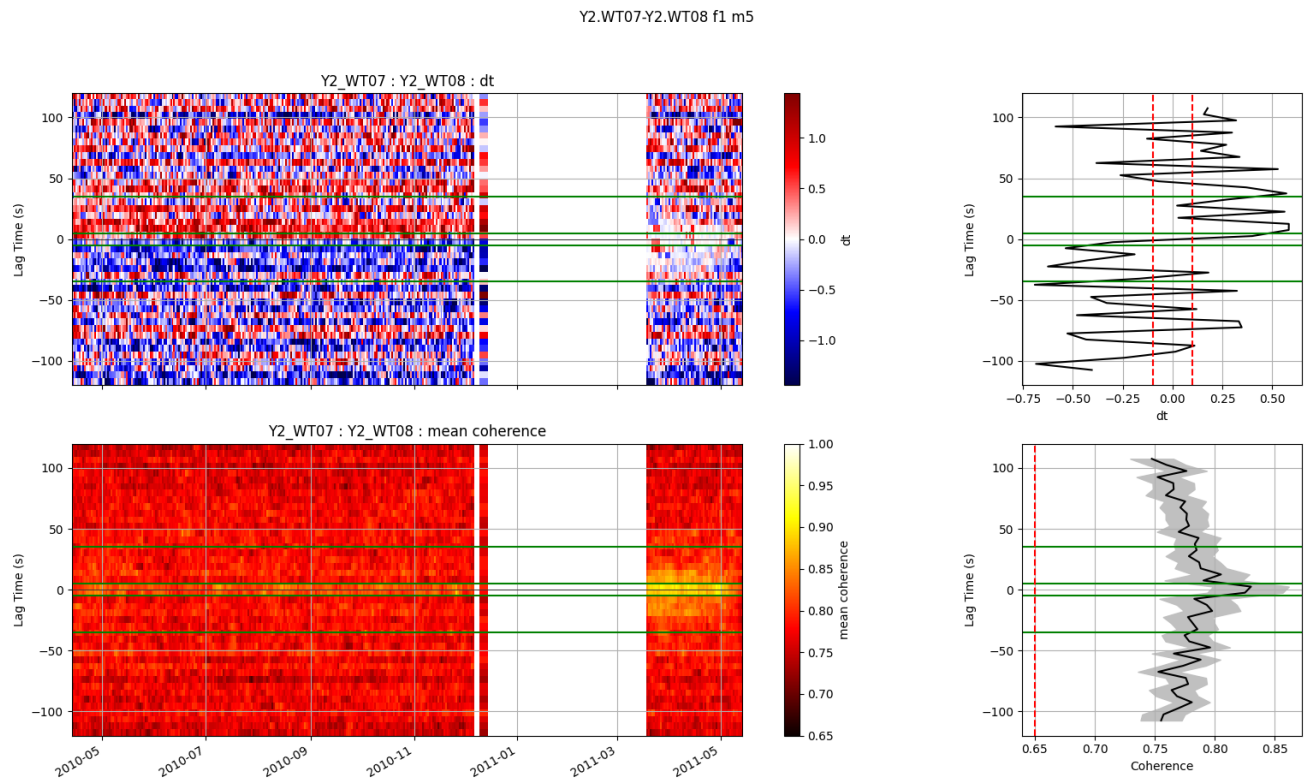


\$ ms plot distance

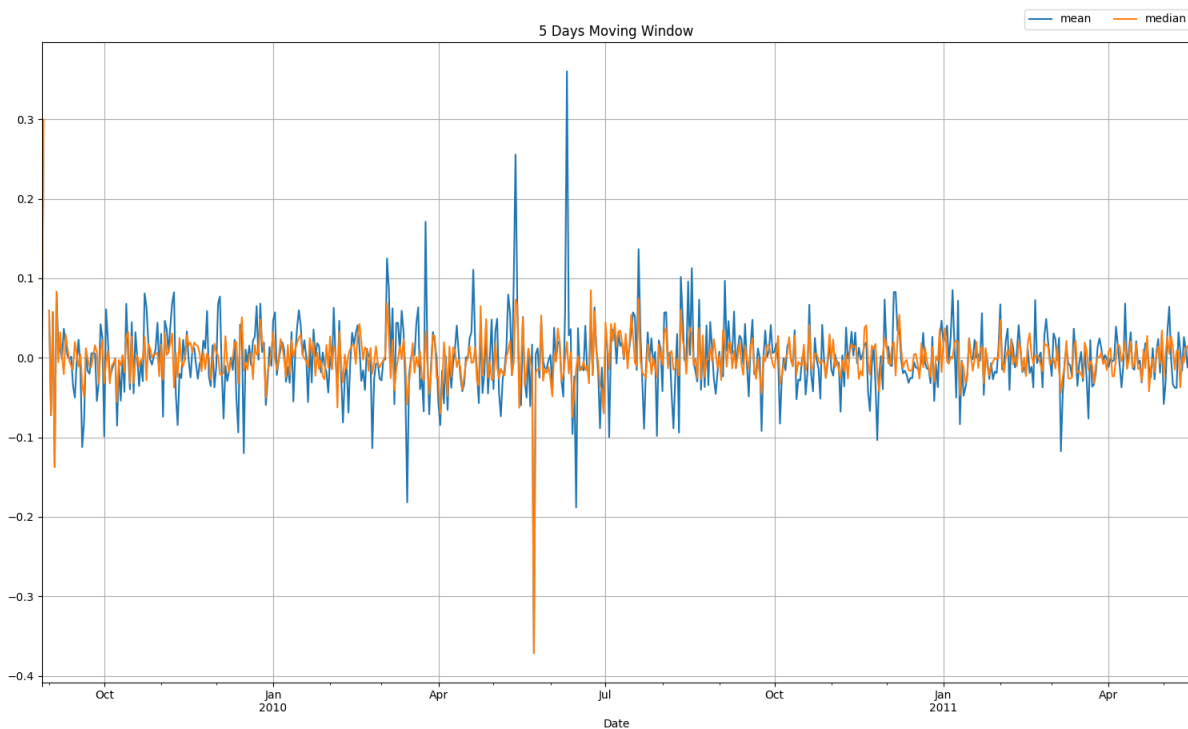


\$ ms plot mwcs Y2.WT07 Y2.WT08 -m 5

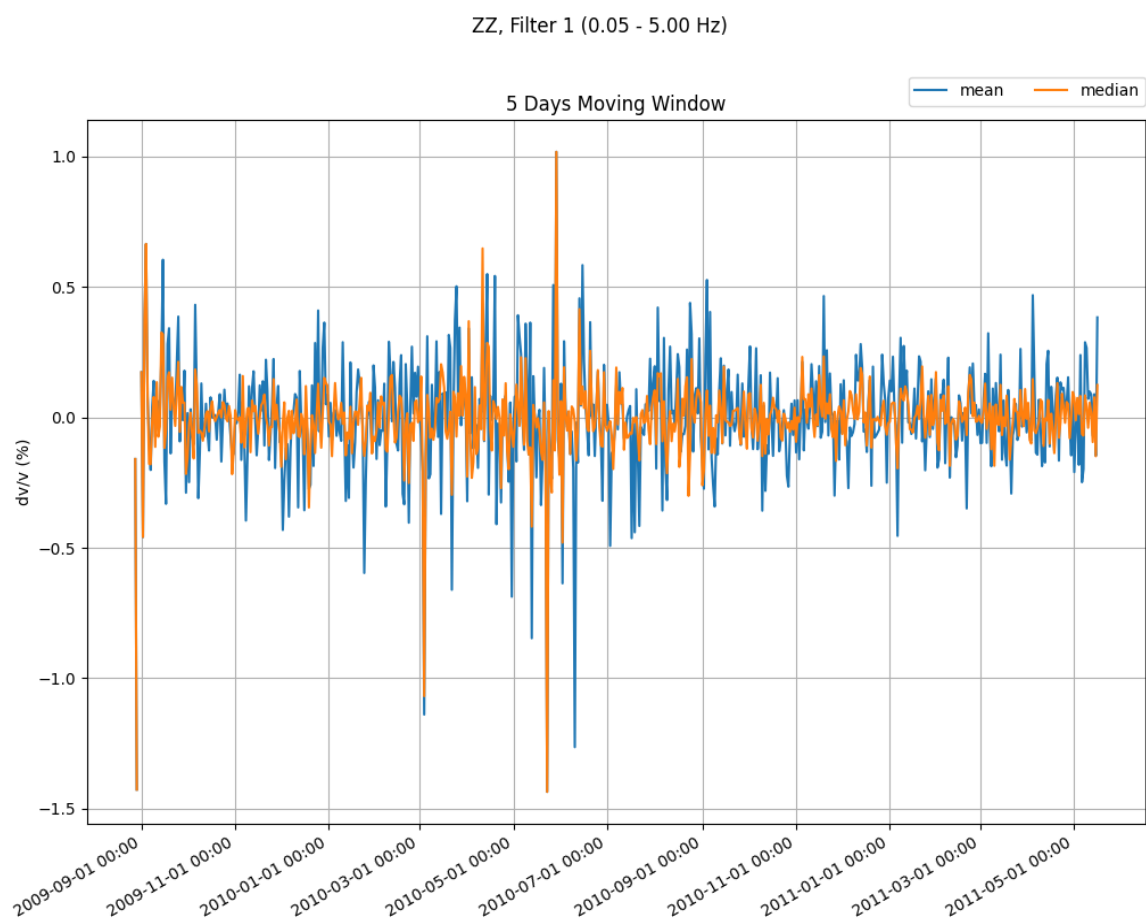
--m parameter passes the moving window length. By default, its value is set to 1 day. However, I have executed the computations for 5-day stacks.



\$ ms plot timing



\$ ms plot dvv



Sources

- MSNoise Documentation
<https://www.msnoise.org/doc/>
- ROBelgium MSNoise Discussion on GitHub
<https://github.com/ROBelgium/MSNoise/discussions>
- MSNoise, a Python Package for Monitoring Seismic Velocity Changes Using Ambient Seismic Noise (2014)
by Thomas Lecocq, Corentin Caudron, and Florent Brenguier
https://www.researchgate.net/publication/262197891_MSNoise_a_Python_Package_for_Monitoring_Seismic_Velocity_Changes_Using_Ambient_Seismic_Noise
- Investigations of Ambient Noise Velocity Variations in a Region of Induced Seismicity near Greeley, Colorado (2023)
by Tom Clifford, Anne Sheehan, and Morgan P. Moschetti
https://www.researchgate.net/publication/367406909_Investigations_of_Ambient_Noise_Velocity_Variations_in_a_Region_of_Induced_Seismicity_near_Greeley_Colorado